

HASHİNG BASICS

[Kaynak](#)

İÇERİK

1. Hash Fonksiyonları: Verinin Dijital Parmak İzi
2. Kimlik Doğrulama ve Güvensiz Parola Saklama Yöntemleri
3. Parolaların Güvenli Saklanması: Hashing, Rainbow Tables ve Salting
4. Hash Tanıma: Hash Türlerini Nasıl Tespit Ederiz?
5. Parola Kırma (Password Cracking): Teknikler ve Araçlar
6. Bütünlük Kontrolü İçin Hashing (Integrity Checking)
7. Kriptografinin Büyük Resmi ve Kritik Ayrım

Hash Fonksiyonları: Verinin Dijital Parmak İzi

Kriptografinin en temel araçlarından biri olan **Hash (Özet)** fonksiyonları, şifrelemeden (encryption) çok farklı bir mantıkla çalışır. En önemli farkı, **tek yönlü (one-way)** olmasıdır; yani elinizde bir özet (digest) varsa, bu özeti geri çevirip orijinal veriye ulaşmak matematiksel olarak imkansız (veya pratik olarak imkansız) kabul edilir.

1. Hash Fonksiyonu Nedir?

Hash fonksiyonu, herhangi bir boyuttaki veriyi (bir harf, bir kitap veya bir disk imajı) alıp, onu **sabit uzunlukta** bir çıktıya (özet) dönüştüren matematiksel bir algoritmadır.

Temel Özellikleri:

- **Sabit Uzunluk:** Girdi ne kadar büyük olursa olsun, çıktı her zaman aynı uzunluktaadır (Örn: SHA-256 her zaman 256 bit üretir).
- **Tek Yönlülük:** Girdiden çıktıya gitmek saniyelik bir iştir, ancak çıktıdan girdiyi bulmak imkansızdır.
- **Hız:** Hesaplaması hızlıdır.
- **Çığ Etkisi (Avalanche Effect):** Girdideki tek bir bitlik değişim bile, çıktının tamamen farklı görünmesine neden olur.
- **Tahmin Edilemezlik:** Çıktıya bakarak girdinin ne olduğunu tahmin edemezsiniz.

Çığ Etkisi Örneği (Teknik Analiz)

Aşağıdaki örnekte `file1.txt` (T harfi) ve `file2.txt` (U harfi) dosyalarının hash değerlerini karşılaştıralım. Binary düzeyde T (**01010100**) ve U (**01010101**) arasında sadece **1 bitlik** fark vardır.

```
strategos@g5000 ~> md5sum *.txt
b9ece18c950afbfa6b0fdbfa4ff731d3  file1.txt
4c614360da93c0a041b22e537de151eb  file2.txt

strategos@g5000 ~> sha256sum *.txt
e632b7095b0bf32c260fa4c539e9fd7b852d0de454e9be26f24d0d6f91d069d3  file1.txt
a25513c7e0f6eaa80a3337ee18081b9e2ed09e00af8531c8f7bb2542764027e7  file2.txt
```

Gördüğün gibi, sadece 1 bit değişmesine rağmen hash değerleri arasında hiçbir benzerlik kalmadı. Bu, verinin bütünlüğünü kontrol etmek için mükemmel bir özelliktir.

2. Hashing Neden Önemlidir?

Hashing, modern güvenliğin iki ana alanında gizlice ama yoğun bir şekilde kullanılır:

1. **Veri Bütünlüğü (Integrity):** İndirdiğin bir dosyanın yolda bozulup bozulmadığını veya bir saldırgan tarafından değiştirilip değiştirilmediğini anlamak için hash değerine bakarsın.
2. **Parola Güvenliği:** İyi bir sistem, parolanı asla açık metin (**cleartext**) olarak veritabanına kaydetmez. Parolanın **hash değerini** saklar. Sen giriş yaparken girdiğin parolanın hash'i alınır ve veritabanındaki ile karşılaştırılır. Böylece veritabanı çalınsa bile saldırganlar doğrudan parolalara ulaşamaz.

3. Hash Collision (Hash Çakışması) Nedir?

Hash Collision, iki farklı girdinin aynı hash çıktısını üretmesi durumudur.

Güvercin Yuvası İlkesi (Pigeonhole Effect)

Girdi kümesi sonsuzdur (istediğin boyutta veri yükleyebilirsin), ancak çıktı kümesi sınırlıdır (örneğin MD5 sadece 2^{128} farklı sonuç üretebilir). Bu durumda matematiksel olarak bazı farklı girdilerin aynı sonucu vermesi **kaçınılmazdır**.

Örnek: Eğer sadece 16 farklı kutun (hash sonucun) varsa ama elinde 21 tane güvercinin (girdi verin) varsa, en az bir kutuda birden fazla güvercin olmak zorundadır.

Güvenlik Riski

Modern bir hash algoritmasının güvenliği, bir "çakışma mühendisliği" (intentional collision) yapmanın ne kadar zor olduğuyla ölçülür.

- **MD5 ve SHA1:** Artık **GÜVENSİZ** kabul edilirler. Araştırmacılar, farklı içeriklere sahip iki dosyanın aynı MD5/SHA1 hash'ini vermesini sağlayacak saldırılar geliştirmişlerdir (**Shattered** saldırısı gibi).
- **Öneri:** Parola saklama veya kritik veri doğrulama için **SHA-256** veya **SHA-512** (SHA-2 ailesi) veya daha iyisi **SHA-3** kullanılmalıdır.

Teknik Notlar ve Formatlar

Hash fonksiyonu çıktıları aslında ham byte dizileridir. Ancak biz bunları okuyabilmek için genellikle **Hexadecimal** (onaltılık) veya **Base64** olarak kodlarız.

- `md5sum` , `sha1sum` gibi Linux araçları her zaman **hexadecimal** çıktı verir.
- Her bir byte, iki hexadecimal hane ile temsil edilir (Örn: 256-bit hash = 64 karakterli hex çıktısı).

Kimlik Doğrulama ve Güvensiz Parola Saklama Yöntemleri

Kriptografinin siber güvenlikteki en kritik uygulama alanlarından biri **parola saklama** (password storage) işlemleridir. Bir sistemin ne kadar "hacklenebilir" olduğunu, parolaları veritabanında nasıl tuttuğu belirler.

Parola Saklama vs. Parola Yöneticileri

Öncelikle bir ayrımı netleştirelim:

- **Parola Yöneticileri (Password Managers):** Bitwarden veya LastPass gibi araçlar, parolayı size **cleartext** (açık metin) olarak geri vermek zorundadır. Bu yüzden şifreleme (encryption) kullanırlar.
- **Kimlik Doğrulama (Authentication):** Bir web sitesi (örneğin TryHackMe), parolanızın ne olduğunu bilmek zorunda değildir. Sadece sizin o parolayı **bildiğinizi** teyit etmesi yeterlidir. Bu yüzden burada "geri döndürülemez" olan **hashing** kullanılır.

Güvensiz Parola Saklama Yöntemleri ve Tarihi Dersler

Veritabanı sızıntıları kaçınılmaz olabilir, ancak sızan verinin saldırganın işine yarayıp yaramayacağı uygulanan yönteme bağlıdır. Tarihte üç büyük hata yapılmıştır:

1. Parolaları Açık Metin (Plaintext) Olarak Saklamak

Bu, yapılabilecek en büyük güvenlik hatasıdır. Veritabanı ele geçirildiği an tüm kullanıcıların parolaları saldırganın eline geçer.

- **Vaka Analizi (RockYou - 2009):** Sosyal medya widget'ları geliştiren RockYou şirketi, 32 milyon kullanıcısının parolasını hiçbir koruma olmadan (plaintext) saklıyordu.
- **Sonuç:** Veritabanı sızdığında 14 milyondan fazla benzersiz parola internete düştü. Bugün sızma testlerinde kullandığımız meşhur **rockyou.txt** listesinin kaynağı bu sızıntıdır.

Kali Linux'ta bu listeye `/usr/share/wordlists/rockyou.txt` yolundan ulaşabilir, `wc -l` komutuyla satır sayısını görebilirsin.

2. Eski/Zayıf Şifreleme Algoritmaları Kullanmak

Parolaları hash'lemek yerine, anahtarı olan bir algoritma ile şifrelemek (encryption) hatadır. Çünkü anahtar ele geçirilirse tüm parolalar geri döndürülür.

- **Vaka Analizi (Adobe - 2013):** Adobe, parolaları hash'lemek yerine artık güvenli olmayan (deprecated) bir şifreleme formatı ile saklıyordu.
- **Hata:** Parola ipuçlarını da açık metin olarak saklamışlardı. Bu durum, saldırganların şifreleme anahtarına gerek duymadan milyonlarca parolayı hızla tahmin etmesine yol açtı.

3. Güvensiz Hash Fonksiyonları Kullanmak

Sadece hash'lemek yetmez; kullanılan algoritmanın çakışmalara ve kaba kuvvet saldırılarına dirençli olması gerekir.

- **Vaka Analizi (LinkedIn - 2012):** LinkedIn, kullanıcı parolalarını **SHA-1** algoritması ile saklıyordu.
- **Kritik Hata (No Salt):** LinkedIn "Salt" (tuzlama) yöntemini kullanmamıştı.
- **Salt (Tuzlama) Nedir?** Parola hash'lenmeden önce sonuna eklenen rastgele bir değerdir. Eğer tuzlama yapılmazsa, aynı parolayı kullanan binlerce kişinin hash'i veritabanında aynı görünür. Bu da saldırganların **Rainbow Tables** (önceden hesaplanmış hash tabloları) ile parolaları saniyeler içinde kırmasını sağlar.

Teknik Özet ve Çıkarılan Dersler

Bir sızma testinde bir veritabanı dump'ı ele geçirdiğinde şunlara bakmalısın:

1. **Algoritma Tipi:** Hash'ler MD5 mi, SHA-1 mi yoksa modern bir standart olan **bcrypt/Argon2** mi?
2. **Salt Var mı?** Aynı değere sahip birden fazla hash varsa, o sistemde tuzlama (salting) yapılmamış demektir. Bu, kırma (cracking) işlemini çok hızlandırır.
3. **Plaintext Sızıntısı:** Eğer veritabanında parolalar doğrudan okunabiliyorsa, bu sadece o sistemin değil, kullanıcıların aynı parolayı kullandığı diğer tüm platformların (banka, e-posta vb.) tehlikede olduğu anlamına gelir.

Parolaların Güvenli Saklanması: Hashing, Rainbow Tables ve Salting

Kimlik doğrulama sistemlerinde temel kural şudur: **Parolayı değil, parolanın hash değerini sakla.** Ancak sadece hash kullanmak, gelişmiş saldırı tekniklerine karşı tek başına yeterli değildir.

1. Hashing Tek Başına Neden Yetersiz? (Rainbow Tables Tehdidi)

Hash fonksiyonları deterministiktir; yani aynı girdi her zaman aynı çıktıyı verir. Eğer iki kullanıcı "123456" parolasını seçerse, veritabanında ikisinin de hash değeri aynı görünür. Bu durum iki büyük risk doğurur:

1. **Toplu İfşa:** Bir hash kırıldığında, aynı parolayı kullanan tüm hesaplar ele geçirilmiş olur.
2. **Rainbow Tables (Gökkuşağı Tabloları):** Milyarlarca popüler parolanın ve bunlara karşılık gelen hash değerlerinin önceden hesaplanıp saklandığı devasa arama tablolarıdır.

Rainbow Tables Mantığı

Bu tablolar, "zaman-alan takası" (**time-memory trade-off**) prensibiyle çalışır. Saldırgan, hash'i kırmak için CPU gücü harcamak yerine, elindeki devasa listede bir "arama" (lookup) yapar.

- **Hız:** CrackStation gibi siteler, bu tablolar sayesinde "tuzlanmamış" (unsalted) hash'leri milisaniyeler içinde çözer.
- **Zayıflık:** Sadece tablodaki önceden hesaplanmış parolaları bulabilir.

2. Rainbow Tablolarına Karşı Savunma: Salting (Tuzlama)

Rainbow tablolarını ve toplu kırma saldırılarını etkisiz hale getirmek için **Salt** (tuz) kullanılır.

Salt Nedir?

Her kullanıcı için rastgele oluşturulan ve parolanın başına veya sonuna eklenen benzersiz bir karakter dizisidir.

Nasıl Çalışır?

Aynı parolayı ("123456") kullanan iki kullanıcı düşünelim:

- **Kullanıcı A:** Parola + Salt1 ("123456@aB1") → Hash A
- **Kullanıcı B:** Parola + Salt2 ("123456 #xY9 ") → Hash B

Gördüğün gibi, parolalar aynı olsa bile **tuzlar farklı olduğu için** veritabanındaki hash değerleri tamamen farklı olur. Bu durumda saldırganın her bir kullanıcı için ayrı bir Rainbow Table oluşturması gerekir ki bu da imkansızdır.

Önemli Not: Salt gizli tutulmak zorunda değildir, veritabanında hash'in hemen yanında saklanabilir. Amacı gizlilik değil, hash'i benzersiz kılmaktır. **Bcrypt** ve **Scrypt** gibi modern algoritmalar bu süreci otomatik yönetir.

3. Güvenli Parola Saklama Adımları (Best Practices)

Bir sistem tasarlarken veya test ederken şu adımlar izlenmelidir:

1. **Güçlü Algoritma Seçimi:** MD5 veya SHA1 yerine; kaba kuvvet saldırılarına karşı yavaşlatılmış (work factor içeren) **Argon2**, **Scrypt**, **Bcrypt** veya **PBKDF2** seçilmelidir.
2. **Benzersiz Tuzlama:** Her kullanıcı için eşsiz bir salt üretilmelidir.
3. **Birleştirme:** Parola ve salt birleştirilir (Concatenation).
4. **Hashing:** Birleştirilen string seçilen algoritma ile hash'lenir.
5. **Depolama:** Hem **hash** hem de **salt** veritabanına kaydedilir.

4. Neden Şifreleme (Encryption) Kullanmıyoruz?

"Neden bu kadar karmaşık süreçlerle uğraşıyoruz, parolaları doğrudan şifreleyip saklasak olmaz mı?" diye düşünebilirsin. Cevap: **Key Management (Anahtar Yönetimi)**.

- **Risk:** Şifreleme (Encryption) çift yönlüdür. Eğer parolaları şifrelersen, bir "şifre çözme anahtarı" saklaman gerekir.
- **Felaket Senaryosu:** Bir saldırgan veritabanıyla birlikte bu anahtarı da ele geçirirse, **tüm** kullanıcıların parolalarını tek bir komutla açık metne (cleartext) dönüştürebilir.
- **Hashing'in Avantajı:** Hashing tek yönlüdür. Ortada bir "anahtar" yoktur. Veritabanı sızsa bile, orijinal parolaya ulaşmak için her bir hash'in tek tek kırılması gerekir (bu da çok maliyetli ve zaman alıcıdır).

Hash Tanıma: Hash Türlerini Nasıl Tespit Ederiz?

Ofansif güvenlik perspektifinden, elimizde bir hash varsa ilk adımımız bunun hangi algoritma ile üretildiğini anlamaktır. Yanlış algoritma seçimi, kırma (cracking) sürecinde zaman kaybına yol açar. **hashID** gibi otomatik araçlar olsa da, bunlar bazen yanılabılır; bu yüzden yapıyı manuel olarak analiz edebilmek kritiktir.

1. Linux Parola Hashleri (/etc/shadow)

Linux sistemlerde kullanıcı parolaları `/etc/shadow` dosyasında saklanır. Bu dosya sadece **root** yetkisiyle okunabilir. Her satır 9 alandan oluşur ve alanlar `:` ile ayrılır. En önemli kısım ikinci alandaki hash yapısıdır:

Format: `$prefix$options$salt$hash`

Prefix (Algoritma Tanımlayıcıları)

Hash'in en başındaki karakterler hangi algoritmanın kullanıldığını söyler. Güçlünden zayıfa doğru yaygın prefixler:

Prefix	Algoritma	Notlar
<i>y</i>	yescrypt	Modern sistemlerde varsayılan ve en güçlü seçenek.
<i>7</i>	scrypt	Bellek yoğunluklu, kırması zor bir fonksiyon.
<i>2b, 2y</i>	bcrypt	Blowfish tabanlı. BSD ve modern Linuxlarda yaygın.

Prefix	Algoritma	Notlar
6	sha512crypt	SHA-512 tabanlı. Eski ama hala çok yaygın.
1	md5crypt	MD5 tabanlı. Günümüz standartlarında zayıf.

Örnek Analiz:

strategos:\$y\$j9T\$76UzfgEM5PnymhQ7TLJey1\$/00Sg64...

- **y**: Algoritma **yescrypt**.
- **9**: Algoritma parametreleri (seçenekler).
- **76Uzfg...**: Bu kullanıcıya özel **Salt** değeri.
- **/00Sg64...**: Gerçek **Hash** çıktısı.

2. Windows Parola Hashleri (NTLM)

Windows, parolaları **SAM** (Security Accounts Manager) veritabanında saklar. Windows'ta kullanılan temel format **NTLM**'dir (MD4 varyantı).

- **Görünüm**: NTLM hashleri 32 karakterlik hexadecimal dizilerdir.
- **Kritik Uyarı**: Görsel olarak MD4 ve MD5 ile birebir aynıdır.
- **Bağlam (Context) Önemlidir**: Eğer bir Windows makineden (SAM veya bellekten mimikatz ile) dump aldıysan, bu hash %99 **NTLM**'dir, MD5 değildir.
- **Yapı**: Genellikle **LM Hash** : **NT Hash** şeklinde ikili yapıda karşımıza çıkar (Modern sistemlerde LM kısmı genellikle boştur veya standart bir değerdir).

3. Hash Tanıma Stratejileri

Otomatik araçlar (hashID, name-that-hash) bazen bir hash için 5-6 farklı ihtimal verebilir. Bu durumda şu kriterleri kullanmalısın:

1. Uzunluk:

- 32 karakter (Hex) -> MD5 veya NTLM
- 40 karakter (Hex) -> SHA-1
- 64 karakter (Hex) -> SHA-256

2. Karakter Seti: Sadece 0-9 ve A-F arası ise Hexadecimal; / , + ve = içeriyorsa Base64 kodlanmış olabilir.

3. Prefix: Başında \$ işareti olan bir yapı görüyorsan Linux/Unix stili bir hash ile karşı karşıyasındır.

4. Kaynak: Hash'i nereden buldun? Veritabanından mı (PHP/MySQL ise MD5 yaygındır), bir sistem dosyasından mı (/etc/shadow), yoksa bir web trafiğinden mi?

Teknik İpucu: Hashcat Örnekleri

Bir hash'in türünden emin değilsen, [Hashcat Example Hashes](#) sayfası en iyi referans kaynağıdır. Oradaki örneklerle elindeki hash'in uzunluğunu ve formatını karşılaştırarak doğru modu (Mode ID) bulabilirsin.

Parola Kırma (Password Cracking): Teknikler ve Araçlar

Parola hash'lerini "deşifre edemezsiniz" çünkü hash'leme çift yönlü bir işlem değildir. Parola kırma işlemi aslında bir **deneme-yanılma** sürecidir: Binlerce olası parolayı (ve varsa tuzu) aynı algoritma ile hash'leyip, elimizdeki hedef hash ile eşleşip eşleşmediğine bakarsınız.

1. Donanım Gücü: CPU vs. GPU

Hash kırma hızı, saniyede kaç hash denenebildiğine (H/s) bağlıdır.

- **GPU (Grafik İşlemci):** Binlerce çekirdeğe sahiptir. Basit ama çok sayıda matematiksel işlemi aynı anda yapabildiği için **MD5, SHA-1, NTLM** gibi algoritmaları kırmada CPU'dan kat kat hızlıdır.
- **CPU (Merkezi İşlemci):** Daha az çekirdeğe sahiptir ama daha karmaşıktır. **John the Ripper** varsayılan olarak CPU kullanır.
- **Dirençli Algoritmalar:** **Bcrypt** gibi bazı algoritmalar, GPU hızlandırmasına dirençli olacak şekilde (yüksek bellek veya özel döngülerle) tasarlanmıştır. Bu tür hash'lerde GPU kullanmak ciddi bir avantaj sağlamaz.

2. Sanallaştırma ve Performans

Sızma testlerinde sıklıkla Sanal Makine (VM) kullansak da, konu parola kırma olduğunda bazı kısıtlamalar vardır:

- **GPU Erişimi:** VM'ler genellikle ana makinenin ekran kartına doğrudan erişemez. Bu yüzden **Hashcat** gibi GPU gücü isteyen araçları doğrudan ana işletim sisteminizde (Host) çalıştırmak her zaman daha verimlidir.
- **CPU Overhead:** VM üzerinden CPU kullanmak, sanallaştırma katmanı nedeniyle az da olsa performans kaybına yol açar. En yüksek hız için kırma işlemlerini ana makinede yapın.

3. Temel Araçlar ve Kullanımı

A. Hashcat (GPU Odaklı)

Sektör standardıdır. Çok fazla parametre ve mod desteği sunar.

Temel Sözdizimi:

Bash


```
hashcat -m <hash_modu> -a <saldırı_modu> hash_dosyası sözlük_dosyası
```

- **-m (Hash Type):** Hash türünün sayısal kodu (Örn: 0 = MD5, 100 = SHA-1, 1000 = NTLM, 1800 = SHA-512).
- **-a (Attack Mode):** Saldırı tipi. -a 0 (Straight), sözlükteki kelimeleri sırayla dener.
- **Örnek Komut (Bcrypt kırma):** Bash

```
hashcat -m 3200 -a 0 hedef_hash.txt /usr/share/wordlists/rockyou.txt
```

B. John the Ripper (CPU Odaklı)

Kullanımı daha basittir ve hash türünü otomatik tanıma konusunda daha başarılıdır.

Temel Kullanım:

Bash

```
john --format=<format_adı> --wordlist=sözlük_yolu hash_dosyası
```

- **Format Belirtme:** Eğer otomatik tanımazsa `--format=raw-md5` gibi parametreler eklenir.

4. Pratik İpuçları ve Strateji

1. **Önce Sözlük:** Her zaman önce `rockyou.txt` gibi popüler bir sözlükle başlayın (Dictionary Attack).
2. **Online Servisler:** Eğer hash tuzlanmamışsa (unsalted), CrackStation gibi siteler devasa **Rainbow Table**'ları sayesinde sonucu saniyeler içinde verebilir. (Ancak öğrenme sürecinde bunları sona bırakın).
3. **Hash Tanıma:** Kırma işlemine başlamadan önce hash'in uzunluğunu ve prefix'ini kontrol ederek doğru modu seçtiğinizden emin olun. (Hatırlatma: 1800 kodu Linux `/etc/shadow` dosyalarındaki `$6$` ile başlayan SHA-512 hash'leri içindir).

Bütünlük Kontrolü İçin Hashing (Integrity Checking)

Hashing'in parola saklama dışındaki en büyük kullanım alanı **veri bütünlüğünü** (integrity) doğrulamaktır. Verinin tek bir biti bile değişse hash tamamen değiştiği için, bu yöntem "dosya orijinal mi yoksa manipüle mi edildi?" sorusunun en net cevabıdır.

1. Dosya Bütünlük Kontrolü (Checksums)

Büyük dosyalar (ISO imajları, yazılım paketleri vb.) indirirken, dosyanın eksik inmediğinden veya bir saldırgan tarafından içine zararlı yazılım (backdoor) yerleştirilmediğinden emin olmak için **Checksum** (kontrol toplamı) kullanılır.

Örnek Senaryo: Fedora ISO Doğrulaması

Fedora gibi dağıtımlar, yayınladıkları ISO dosyalarının yanında bir de `CHECKSUM` dosyası paylaşırlar. Bu dosya genellikle **PGP** ile imzalanmıştır (böylece hash listesinin kendisinin de orijinal olduğu kanıtlanır).

Plaintext

```
# Fedora-Workstation-Live-x86_64-40-1.14.iso için beklenen hash:  
dd1faca950d1a8c3d169adf2df4c3644ebb62f8aac04c401f2393e521395d613
```

Nasıl Kontrol Edilir?

Terminalde indirdiğin dosya üzerinde şu komutu çalıştırırsın:

Bash

```
sha256sum Fedora-Workstation-Live-x86_64-40-1.14.iso
```

Eğer çıkan sonuç, resmi listedekiyle **karakteri karakterine** aynıysa, dosya %100 orijinaldir.

Pro-Tip: Hashing aynı zamanda **Duplicate File** (kopya dosya) tespiti için de kullanılır. Dosya isimleri farklı olsa bile hash değerleri aynıysa, o dosyalar birbirinin aynısıdır.

2. HMAC (Keyed-Hash Message Authentication Code)

Standart bir hash (Örn: SHA256), verinin değişip değişmediğini söyler ama veriyi **kimin** gönderdiğini kanıtlamaz. Eğer bir saldırgan hem mesajı değiştirip hem de yeni mesajın hash'ini hesaplayıp size gönderirse, ruhunuz bile duymaz.

İşte burada **HMAC** devreye girer. HMAC, bir hash fonksiyonunu **gizli bir anahtar** (secret key) ile birleştirir.

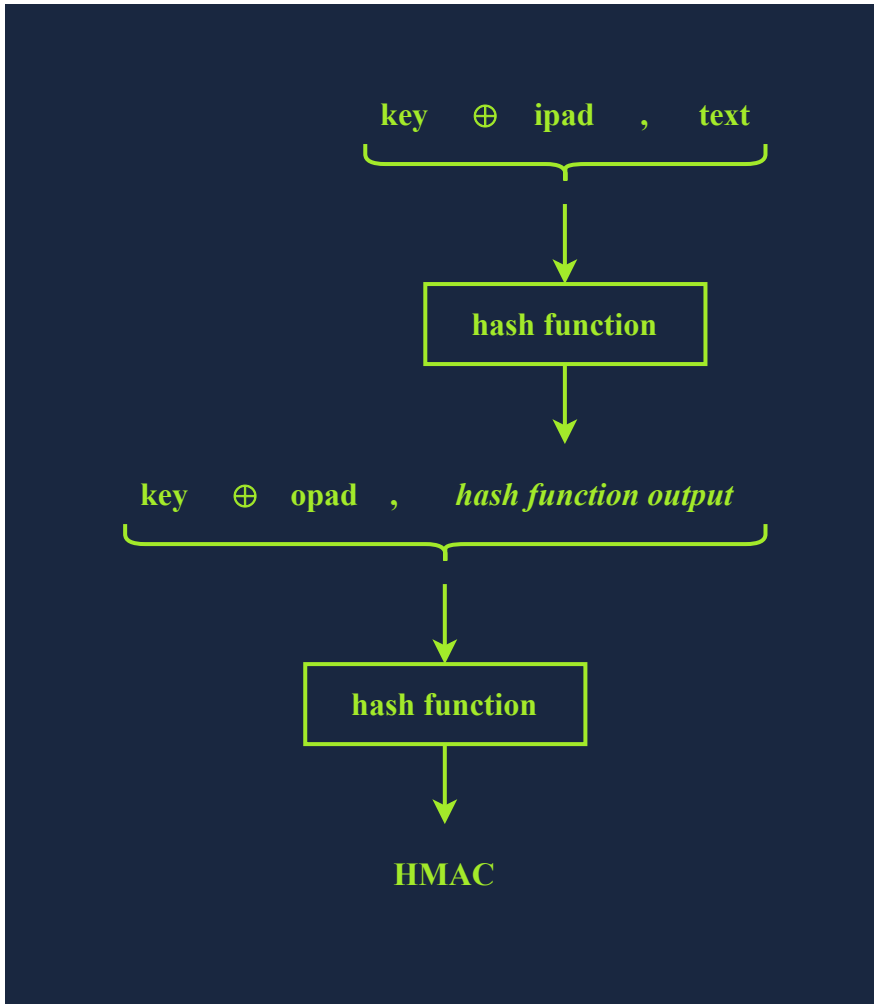
HMAC'in Sağladığı Avantajlar:

- Integrity (Bütünlük):** Verinin değişmediğini kanıtlar.
- Authenticity (Kimlik Doğrulama):** Verinin sadece "gizli anahtara sahip olan kişi" tarafından oluşturulduğunu kanıtlar.

HMAC Nasıl Hesaplanır? (Matematiksel Mantık)

Süreç sadece anahtarı mesajın sonuna eklemekten daha karmaşıktır. İki aşamalı bir hashing işlemi yapılır:

$$AC(K,) = ((K \oplus opad)((K \oplus ipad)))$$



- **M:** Mesaj (Veri)
- **K:** Gizli Anahtar
- **⊕ (XOR):** Önceki derslerde gördüğümüz XOR işlemi.
- **ipad / opad:** İç ve dış "padding" (dolgu) sabitleri.
- **||:** Concatenation (birleştirme).

Özetle Süreç:

1. Anahtar, sabit bir değerle (ipad) XOR'lanır ve mesajla birleştirilip hash'lenir.
2. Çıkan sonuç, anahtarın başka bir sabitle (opad) XOR'lanmış haliyle tekrar birleştirilip bir kez daha hash'lenir.

Neden bu kadar karmaşık?

Sadece `Hash(Key + Message)` yapmak, "Length Extension Attack" gibi bazı kriptografik saldırılara karşı zayıf kalabilir. HMAC'in bu çift katmanlı yapısı bu saldırıları imkansız kılar.

Önemli Notlar

- **Kullanım Alanı:** HMAC'ler özellikle API güvenliğinde (API Key doğrulama), ağ protokollerinde ve dijital imzalarda sıkça kullanılır.
- **Güvenlik:** HMAC'in güvenliği, kullanılan alt hash fonksiyonuna (SHA256, SHA3 vb.) ve anahtarın gizliliğine bağlıdır.

Kriptografinin Büyük Resmi ve Kritik Ayrım

Kriptografi modülümüzün sonuna gelirken, bir siber güvenlik uzmanının en sık yaptığı (ve yapmaması gereken) temel bir karışıklığa son noktayı koyuyoruz. **Hashing**, **Encoding** ve **Encryption** arasındaki farkı bilmek, sadece dersten 100 almak için değil, bir sistemdeki zafiyeti doğru analiz etmek için de hayati önem taşır.

Kutsal Üçlü: Hashing vs. Encoding vs. Encryption

Aşağıdaki tabloyu aylar sonra baktığında "bu neydi?" demeyecek şekilde hafızana kazı:

İşlem	Amaç	Geri Döndürülebilir mi?	Anahtar (Key) Var mı?
Hashing	Veri bütünlüğünü (Integrity) ve parolanın gizliliğini doğrulamak.	HAYIR (Tek yönlüdür).	HAYIR (Sadece veri girer).
Encoding	Veriyi farklı sistemlerin okuyabileceği bir formata dönüştürmek.	EVET (Herkes yapabilir).	HAYIR .
Encryption	Verinin gizliliğini (Confidentiality) sağlamak.	EVET (Sadece yetkililer).	EVET (Olmazsa olmaz).

1. Encoding (Kodlama) Hakkında Bilmen Gerekenler

Encoding bir **güvenlik önlemi değildir**. Sadece verinin iletilmesi veya saklanması için kullanılan bir "çeviri" işlemidir.

- Dil Odaklı Kodlamalar:** ASCII, UTF-8 (en yaygın), UTF-16. Bunlar karakterlerin bilgisayar dilindeki karşılıklarıdır.
- Veri İletim Kodlamaları (Base32/Base64):** Özellikle binary verileri (resim, dosya vb.) metin tabanlı protokoller üzerinden (HTTP, E-posta) göndermek için kullanılır.

Base64 Örneği:

Bash

```
# Kodlama (Encoding)
strategos@g5000 ~> echo "TryHackMe" | base64
VHJ5SGFja01lCg==

# Çözme (Decoding) - Herkes yapabilir!
strategos@g5000 ~> echo "VHJ5SGFja01lCg==" | base64 -d
TryHackMe
```

Dikkat: Bir CTF'te sonu `==` ile biten bir string görüyorsan, bu %90 ihtimalle Base64'tür. Bu bir "şifre" değildir, sadece kodlanmış bir metindir.

2. Özet: Kriptografik Süreçler

- **Hashing:** Girdiden sabit boyutlu bir "özet" (digest) üretir. Geriye döndürülemez. Veri değişirse özet de değişir (Bütünlük kontrolü).
- **Encryption:** Bir **Cipher** (algoritma) ve bir **Key** (anahtar) kullanarak veriyi gizler. Doğru anahtara sahip olan kişi veriyi orijinal haline döndürebilir (Gizlilik).

Bir Sonraki Seviye İçin Yol Haritası

Kriptografi temellerini sağlam attık. Bu bilgileri pratiğe dökmek ve "ofansif" yeteneklerini geliştirmek için şu adımları takip etmelisin:

1. **John the Ripper:** Öğrendiğin hash türlerini gerçek hayatta kırmayı öğrenmek için bu odaya devam et.
2. **Kriptografi Derin Dalış:** RSA, Diffie-Hellman gibi kavramların matematiksel açıklarını ve daha gelişmiş araçları öğrenmek için ileri seviye kriptografi odalarını incele.
3. **Hashcat:** GPU gücünü kullanarak saniyede milyonlarca hash deneme yeteneği kazan.

```
Dataview (inline field '='): Error:
```

```
-- PARSING FAILED -----  
---
```

```
> 1 | =  
    | ^
```

Expected one of the following:

```
('', 'null', boolean, date, duration, file link, list ('[1, 2,  
3]'), negated field, number, object ('{ a: 1, b: 2 }'), string,  
variable
```